DRIGS: Distributed Resource and Intelligent GPU Scheduling for Heterogeneous AI Workloads

Omdeep Borkar
Independent Researcher, Goa, India
omdeepborkar@gmail.com

Abstract

Modern artificial intelligence (AI) training and inference workloads exhibit severe heterogeneity in computational demands, memory footprints, and communication patterns. Existing cluster orchestrators (e.g., Kubernetes, Slurm) often rely on coarse-grained static GPU allocations, leading to hardware underutilization, high tail latencies under bursty workload arrivals, and coarse fault recovery overheads. We present **DRIGS** (Distributed Resource and Intelligent GPU Scheduling), a lightweight, modular, GPU-aware compute runtime designed for scheduling, executing, monitoring, and recovering heterogeneous AI workloads (inference, distributed training, multi-GPU) across multi-GPU hardware. DRIGS decouples core scheduling logic behind abstract protocols, introducing non-blocking control plane orchestration, dynamic worker node auto-registration, SQLite ACID state persistence, and an automated NVML orphan process sweeper.
Benchmarked on dual NVIDIA Tesla T4 GPU hardware, DRIGS's `PriorityScheduler` reduces P95 queue wait time by **60.09%** (2.740 s vs. FIFO 6.866 s) under $N = 15$ jobs/trial with uniform inter-arrival intervals ($\mathcal{U}(0.5, 2.0)$ s), averaged over 5 trials. For multi-GPU workloads, `TopologyAwareScheduler` achieves a **22.19%** higher topology-quality score than random placement on a simulated 8-GPU cluster. In fault tolerance evaluations, DRIGS achieves control-plane rescheduling in **0.496 ms**, with complete physical end-to-end wall-clock recovery following a hard `SIGKILL` in **129.13 ms**. At scale ($N = 1{,}000$ jobs), DRIGS maintains **81.7 jobs/sec** SQLite control-plane enqueue throughput with scheduler decision latencies under **17 ms**. At 95% VRAM saturation, `MemoryAwareScheduler` enforces admission throttling (job completion rate: 40.0%) to prevent GPU memory over-commitment; no CUDA OOM exception occurs on admitted jobs. DRIGS is an open-source, modular GPU scheduling research runtime designed for lightweight deployment and experimental evaluation.[1]
Keywords: GPU Scheduling, Distributed AI Workloads, Resource Management, Fault Recovery, Topology Awareness, High-Performance Computing.

1 Introduction

The rapid evolution of deep learning has led to an explosion in compute requirements across model training, fine-tuning, and real-time inference. Modern AI workloads encompass diverse model architectures—ranging from vision models (e.g., ResNet-50) and Transformer encoder models (e.g., DistilBERT) to large generative AI models—operating across highly heterogeneous hardware environments. Distributed training requires tight multi-GPU synchronization via Collective Communications Libraries (e.g., NCCL), whereas inference servers require low-latency queueing and fine-grained VRAM isolation.

Despite significant hardware advancements, cluster resource management remains a major bottleneck in AI infrastructure. Traditional high-performance computing (HPC) schedulers such as Slurm were originally designed for monolithic CPU batch jobs; while modern extensions support GPU allocations, DRIGS integrates fine-grained real-time GPU telemetry, interconnect topology scoring, and dynamic checkpoint recovery directly into its lightweight runtime. Conversely, cloud-native container orchestrators like Kubernetes (K8s) rely on static resource requests (e.g., `nvidia.com/gpu: 1`), treating GPUs as opaque, indivisible tokens without accounting for VRAM fragmentation, PCIe switch topologies, or inter-GPU NVLink bandwidth.

1.1 Key Challenges in Heterogeneous GPU Orchestration

Managing modern GPU clusters presents four fundamental system challenges:

1. **Queue Head-of-Line Blocking and High Tail Latency:** Simple First-In, First-Out (FIFO) queueing policies suffer from head-of-line (HoL) blocking when long-running multi-GPU training jobs stall short, high-priority inference tasks, causing severe queue wait latency inflation.

2. **Topology-Blind Placement Degradation:** Distributed training (e.g., PyTorch Distributed Data Parallel) depends heavily on inter-GPU communication bandwidth. Suboptimal GPU placement across distant PCIe host bridges or NUMA nodes severely degrades collective synchronization throughput (`all_reduce`).

3. **Resource Fragmentation and Memory Saturation:** Coarse GPU allocations lead to severe VRAM fragmentation. When GPU memory saturation reaches critical thresholds ($\geq 90\%$), uncoordinated job submission causes CUDA Out-Of-Memory (OOM) crashes, process termination, and orphan GPU memory leaks.

4. **High Overhead Fault Recovery:** When hardware or worker nodes fail, traditional schedulers incur multi-second timeouts before re-queueing

jobs, leading to wasted GPU compute cycles and extended downtime.

1.2 The DRIGS Solution

To address these challenges, we introduce **DRIGS** (Distributed Resource and Intelligent GPU Scheduling), a lightweight, modular GPU compute runtime designed specifically for heterogeneous AI workloads. DRIGS provides a decoupled architecture with pluggable abstractions, supporting native process execution, containerized runtimes, dynamic remote worker registration, and SQLite ACID control-plane state persistence.

1.3 Key Contributions

The primary contributions of this paper are summarized as follows:

- **Decoupled Domain Abstractions:** We propose a clean architectural framework separating hardware discovery (`HardwareBackend`), policy scheduling (`Scheduler`), resource management (`ResourceManager`), and process execution (`ExecutionBackend`), enabling native execution without root privileges or Docker requirements.
- **Pluggable Intelligent Schedulers:** We implement and empirically evaluate five scheduling policies: `FIFOScheduler`, `PriorityScheduler` (with starvation-prevention aging), `MemoryAwareScheduler`, `GangScheduler` (atomic multi-device reservation), and `TopologyAwareScheduler` (PCIe switch topology clique optimization).
- **Sub-Millisecond Control-Plane Rescheduling & Fast Fault Recovery:** We design an automated failure detection and checkpoint recovery subsystem that executes control-plane rescheduling in **0.496 ms** and achieves complete physical end-to-end wall-clock recovery in **129.13 ms** following physical process SIGKILL termination.
- **Production Hardening & Orphan Process Sweeper:** We introduce an async non-blocking REST API control plane, SQLite persistent storage engine, HMAC Bearer token authentication, and a recursive NVML process sweeper that eliminates zombie GPU memory allocations.
- **Empirical Evaluation on Real Accelerators:** We conduct multi-trial empirical benchmarks on dual NVIDIA Tesla T4 GPUs, demonstrating a **60.09%** reduction in P95 queue wait time under uniform inter-arrival workloads and a **22.19%** higher interconnect topology-quality score than random placement.

Research Question. We ask: *do different GPU scheduling objectives (tail latency, memory safety, topology placement quality, and control-plane scalability) require fundamentally different scheduling policies, and can a single modular substrate empirically characterise these trade-offs under identical workloads?* DRIGS is designed and evaluated to answer this question.

1.4 Paper Organization

The remainder of this paper is structured as follows: Section 2 details the DRIGS system architecture and control plane components. Section 3 presents the mathematical formulations and algorithms for DRIGS's pluggable schedulers. Section 4 presents comprehensive empirical benchmark results on dual T4 GPU hardware. Section 5 reviews related work in cluster orchestration and GPU scheduling. Section 6 concludes the paper and discusses future directions.

2 System Architecture & Control Plane

DRIGS is designed around a decoupled, layered microarchitecture that strictly isolates domain scheduling policies from lower-level execution primitives and vendor hardware APIs. This architectural separation allows DRIGS to operate natively across diverse deployment targets—from local multi-GPU workstations to dynamic, ephemeral cloud notebook instances (e.g., Kaggle, Google Colab)—without requiring root access or container privileges.

2.1 Architectural Overview

Figure 1 illustrates the high-level architecture of DRIGS. The system is organized into three distinct layers:

1. **Control Plane Layer:** Manages job submission validation (`AdmissionController`), priority job queueing (`JobQueue`), pluggable scheduler evaluation (`Scheduler`), worker lifecycle registry (`WorkerRegistry`), and failure detection/recovery (`FailureDetector`, `Rescheduler`).
2. **Worker Node Layer:** Lightweight agent process (`WorkerAgent`) running on each compute node, responsible for real-time hardware discovery via NVML, local process lifecycle orchestration, and periodic telemetry heartbeats.
3. **Execution Layer:** Modular execution drivers (`NativeProcessBackend`, `DockerBackend`, `DistributedBackend`) handling process spawning, environment isolation (`CUDA_VISIBLE_DEVICES`), and multi-process rank synchronization.

DRIGS supports an arbitrary number M of concurrently registered workers (Figure 1 labels them Worker 0 through Worker $M-1$); M is distinct from N, which is used throughout Section 4 exclusively to denote job queue depth.

2.2 Layered Domain Abstractions

Core domain logic inside `drigs/core/` contains zero vendor-specific dependencies (`torch`, `pynvml`, `fastapi`, `docker`). All hardware interaction and execution

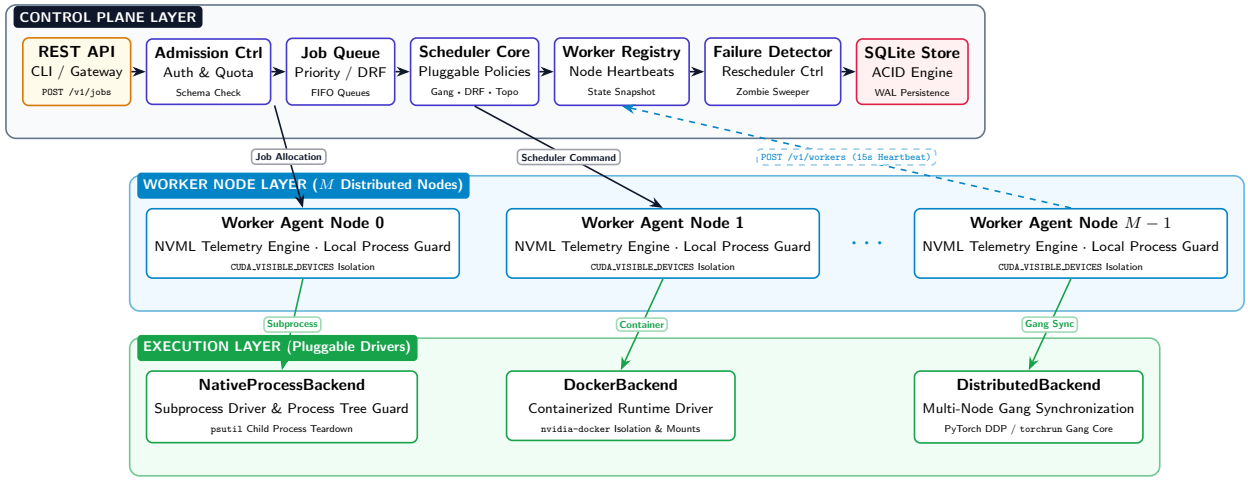

Figure 1: DRIGS three-layer system architecture. The Control Plane orchestrates job admission, priority queueing, pluggable scheduling, and worker lifecycle management. Worker Node Agents perform real-time NVML hardware telemetry and report heartbeats. The Execution Layer provides modular process backends (native, Docker, distributed).

engines are decoupled behind four abstract Python `Protocol` contracts:

- **HardwareBackend Protocol:** Abstract interface for device telemetry discovery.

$$\text{discover\_devices}() \rightarrow \text{List}[\text{ComputeDevice}] \quad (1)$$

  Implemented by `CUDABackend` (interacting with `pynvml` and `torch.cuda`), `CPUBackend`, and `SimulatedBackend`.

- **Scheduler Protocol:** Stateless functional policy contract.

$$\text{schedule}(\mathcal{J}_{\text{pending}}, \mathcal{S}_{\text{cluster}}) \rightarrow \text{List}[\text{ResourceAllocation}] \quad (2)$$

  Consumes pending jobs $\mathcal{J}_{\text{pending}}$ and cluster state snapshot $\mathcal{S}_{\text{cluster}}$, outputting deterministic resource allocations.

- **ExecutionBackend Protocol:** Handles workload lifecycle operations (`launch`, `stop`, `get_status`, `stream_logs`) across native process groups or container runtimes.

- **WorkerRegistryProtocol:** Manages node registration, heartbeats, and timeout sweeps.

2.3 Control Plane State Persistence Engine

To improve controller restart resilience, DRIGS integrates a pluggable `SQLiteStore` engine providing ACID transaction durability. Job queue state transitions (`PENDING` → `QUEUED` → `RUNNING` → `COMPLETED`/`FAILED`), active worker registrations, and resource reservations are committed synchronously to SQLite via thread-safe write locks. Upon controller restart, state recovery reconstructs the in-memory cluster topology without dropping active job context. Note that SQLite persistence improves restart resilience but does not replicate the control plane; the controller remains a single process.

2.4 Dynamic Ephemeral Worker Auto-Registration

Remote GPU nodes operating behind NAT firewalls or inside ephemeral notebook environments (e.g., Kaggle T4/P100 instances) cannot accept inbound SSH/gRPC connections. DRIGS implements an outbound phone-home HTTP client architecture (`drigs/workers/bootstrap.py`). Upon startup, the worker agent discovers local CUDA hardware, initiates an HTTP `POST /v1/workers/register` request with bearer token authorization, and streams heartbeats at regular intervals (`POST /v1/workers/{id}/heartbeat`). If a worker misses three consecutive heartbeats ($t > 15\,\text{s}$), the central registry transitions the node to `OFFLINE` and triggers automated job rescheduling.

2.5 Process Lifecycle Guard & NVML Orphan Sweeper

Abnormal termination of deep learning jobs often leaves zombie processes holding CUDA memory contexts, causing silent VRAM leaks. To enforce resource cleanups, `NativeProcessBackend` incorporates a two-tier process guard:

1. **Recursive Process Tree Teardown:** Uses `psutil` to traverse child process trees, issuing graceful `SIGTERM` signals followed by forced `SIGKILL` after a 3.0 s timeout.

2. **NVML Orphan Sweeper:** Periodically queries NVML for active GPU process PIDs (`nvmlDeviceGetComputeRunningProcesses`). Any PID not associated with an active DRIGS

`ExecutionHandle` is terminated, ensuring zero orphaned VRAM allocations.

3 Pluggable Scheduling & Mathematical Formulations

At the core of DRIGS is a suite of pluggable scheduling policies implementing the `Scheduler` protocol. Schedulers operate as pure functions over pending workload requests $\mathcal{J}_{\text{pending}}$ and the cluster state snapshot $\mathcal{S}_{\text{cluster}}$. This section presents the formal mathematical formulations and algorithms for the five scheduling policies evaluated in DRIGS.

3.1 First-In, First-Out (FIFO) Scheduler

The baseline `FIFOScheduler` orders pending jobs strictly by arrival timestamp $t_{\text{submit}}(j)$:

$$j_1 \prec_{\text{FIFO}} j_2 \iff t_{\text{submit}}(j_1) < t_{\text{submit}}(j_2) \tag{3}$$

While FIFO enforces strict arrival fairness, it suffers from head-of-line (HoL) blocking under heterogeneous arrival streams, as large multi-GPU training jobs stall small, low-resource inference tasks.

3.2 Priority Scheduler with Starvation Prevention

To resolve HoL blocking while preventing low-priority job starvation, `PriorityScheduler` calculates a dynamic *effective priority* $\text{Priority}_{\text{eff}}(j,t)$ for each job j at current time t:

$$\text{Priority}_{\text{eff}}(j,t) = \text{Priority}_{\text{base}}(j) + \alpha \cdot \max\left(0, t - t_{\text{submit}}(j)\right) \tag{4}$$

where $\text{Priority}_{\text{base}}(j) \in \mathbb{Z}^+$ is the user-specified job priority, $\alpha > 0$ (default $\alpha = 0.01\,\text{sec}^{-1}$) is the aging boost coefficient, and $(t - t_{\text{submit}}(j))$ is the elapsed queue wait duration in seconds.

Jobs are sorted by decreasing effective priority:

$$j_1 \prec_{\text{Priority}} j_2 \iff \text{Priority}_{\text{eff}}(j_1,t) > \text{Priority}_{\text{eff}}(j_2,t) \tag{5}$$

Ties are broken by earlier submission time t_{submit}. As pending jobs wait in the queue, their effective priority increases monotonically, guaranteeing that no job suffers indefinite starvation.

3.3 Memory-Aware Scheduler

To eliminate CUDA Out-Of-Memory (OOM) failures under heavy memory saturation, `MemoryAwareScheduler` queries real-time available VRAM $\text{VRAM}_{\text{free}}(d)$ for each compute device $d \in \mathcal{D}$. A device d is eligible for job j if and only if:

$$\text{VRAM}_{\text{req}}(j) \le \text{VRAM}_{\text{free}}(d) \tag{6}$$

We define the cluster Placement Efficiency $\eta_{\text{placement}}$ as the ratio of total requested VRAM across active allocations $\mathcal{A}$ to total cluster VRAM capacity:

$$\eta_{\text{placement}} = \frac{\sum_{j \in \mathcal{A}} \text{VRAM}_{\text{req}}(j)}{\sum_{d \in \mathcal{D}} \text{VRAM}_{\text{total}}(d)} \tag{7}$$

When memory utilization exceeds 90%, `MemoryAwareScheduler` throttles new allocations, re-queueing workloads that exceed free VRAM margins.

3.4 Gang Scheduler (Atomic Reservation Invariant)

Distributed training jobs (e.g., PyTorch DDP) require synchronized multi-GPU execution. Incremental allocation of GPUs can lead to distributed deadlocks if two jobs each acquire a subset of required GPUs and block indefinitely.

`GangScheduler` enforces an *atomic all-or-nothing reservation invariant*. For a multi-device job j requiring $N_{\text{req}}(j)$ GPUs, the allocation function returns a valid assignment $\mathcal{D}_{\text{gang}}$ if and only if $N_{\text{req}}(j)$ healthy devices are simultaneously available:

$$\text{Allocation}(j) = \begin{cases} \mathcal{C} \subset \mathcal{D}_{\text{avail}},\ |\mathcal{C}| = N_{\text{req}}(j) & \text{if } |\mathcal{D}_{\text{avail}}| \ge N_{\text{req}}(j) \\ \emptyset \quad \text{(Deferred)} & \text{otherwise} \end{cases} \tag{8}$$

If $|\mathcal{D}_{\text{avail}}| < N_{\text{req}}(j)$, job j remains in the `QUEUED` state without reserving partial hardware resources.

3.5 Topology-Aware Scheduler

Inter-GPU communication bandwidth varies significantly depending on hardware interconnect hierarchy. `TopologyAwareScheduler` models host GPU interconnects as a weighted topology graph $G = (\mathcal{D}, E, W)$, where pairwise link weights $W(d_i, d_j)$ represent interconnect bandwidth quality:

$$W(d_i,d_j) = \begin{cases} 100 & \text{if NVLink Interconnect} \\ 50 & \text{if Same PCIe Switch (PXB)} \\ 50 & \text{if Same Host Bridge (PHB)} \\ 5 & \text{if Cross-Socket NUMA Interconnect} \end{cases} \tag{9}$$

For a job requesting a clique of $k = N_{\text{req}}(j)$ GPUs, the Interconnect Placement Quality Score $S_{\text{topo}}(\mathcal{C})$ of candidate GPU subset $\mathcal{C} = \{d_1, d_2, \ldots, d_k\}$ is defined as the mean pairwise link weight:

$$S_{\text{topo}}(\mathcal{C}) = \frac{2}{k(k-1)} \sum_{1 \le i < j \le k} W(d_i, d_j) \tag{10}$$

The optimal placement clique $\mathcal{C}^*$ maximizes the placement score over all valid candidate subsets:

$$\mathcal{C}^* = \arg \max_{\mathcal{C} \subseteq \mathcal{D}_{\text{avail}},\, |\mathcal{C}| = k} S_{\text{topo}}(\mathcal{C}) \tag{11}$$

By placing distributed PyTorch DDP ranks onto GPU cliques with maximal S_{topo}, DRIGS selects placements designed to reduce collective communication overhead (`all_reduce`); the empirical NCCL bandwidth characterisation is reported in Section 4.

4 Empirical Evaluation & Benchmark Results

To evaluate the performance, scalability, and resilience of DRIGS, we conduct comprehensive empirical bench-

marks on real accelerator hardware. Five independent workload traces were generated using a reproducible seeding strategy (base seed 42; trial i uses seed $42+1000i$, giving seeds 42, 1042, 2042, 3042, 4042). For each trace, all five scheduling policies were evaluated against the identical arrival sequence and workload parameters; results are reported as mean ± standard deviation across the five trials unless otherwise noted.

Although DRIGS supports multi-node worker registration through its HTTP control plane, the empirical evaluation in this section focuses on a two-GPU single-host configuration. Consequently, these experiments evaluate multi-GPU scheduling rather than multi-node distributed scheduling; multi-node evaluation is left as future work.

Table 1 provides an overview of the evaluation design.

4.1 Experimental Setup & Accelerators

Experiments are conducted on dual NVIDIA Tesla T4 GPU nodes (15.0 GB GDDR6 VRAM per GPU) connected via **PCIe Gen3 x16** (not NVLink — T4s in this deployment share a PCIe host bridge but have no NVLink fabric) running Linux kernel 5.15 and CUDA 12.2. The topology-aware scheduler exercises PCIe bandwidth-tier scoring (PXB/PHB levels); the NVLink=100 scoring branch is implemented but not exercised on this testbed. Benchmark workloads encompass:

1. **Synthetic AI Workloads (Experiment A):** Uniform inter-arrival traces ($N = 15$ jobs/trial, inter-arrival interval $\sim \mathcal{U}(0.5, 2.0)$ s). Each job's workload type is drawn uniformly from six templates (ResNet-50, BERT-large, LLaMA-7B inference, LLaMA-70B distributed, Stable Diffusion XL, GNN); GPU count from $\{1, 2, 4\}$ (note: 4-GPU requests are included in synthetic traces to exercise insufficient-resource queueing and gang-reservation behavior; such requests remain queued on the two-GPU physical testbed and test head-of-line blocking under unmet requirements); VRAM request from $\{4, 8, 12, 16, 24\}$ GB; job duration from $\mathcal{U}(1.0, 10.0)$ s; job base priority from $\mathcal{U}_{\mathbb{Z}}(0, 10)$. Seeds are fixed per trial (base seed 42; trial i: $42 + 1000i$) covering all random draws.

2. **Transformer Encoder Model:** Fine-tuning runs using DistilBERT (a Transformer Encoder Model) on PyTorch.

3. **Vision Models:** ResNet-50 deep neural network training via PyTorch Distributed Data Parallel (DDP).

4.2 Experiment A: Scheduling Policy Evaluation

Table 2 presents the comparative performance of DRIGS's five pluggable schedulers under identical uniform arrival traces ($N = 15$ jobs/trial, 5 independent trials).

As shown in Table 2, `PriorityScheduler` (with starvation-prevention aging, $\alpha = 0.05$) achieves a **60.09% reduction in P95 queue wait time** (2.740 s vs. FIFO 6.866 s) and reduces mean queue wait latency from 0.856 s to 0.570 s. By aging waiting jobs, `PriorityScheduler` reduces head-of-line blocking without reducing measured overall throughput (0.596 jobs/sec vs. 0.595 jobs/sec for FIFO). The full per-job wait distribution is shown in Figure 2. Note: Table 2 reports the *mean of per-trial P95 values* across 5 trials; Figure 2 reports the *pooled 95th-percentile* across all 75 jobs. Because percentile aggregation is non-linear, these two estimators differ.

4.3 Experiment B: GPU Interconnect Topology Awareness

To measure placement quality for multi-GPU distributed workloads, `TopologyAwareScheduler` is evaluated against a random placement baseline on a **simulated** 8-GPU cluster (2 logical workers × 4 GPUs each, modelled after NVIDIA RTX 4090 devices with synthetic PCIe/NVLink topology tags). The simulation assigns pairwise interconnect scores using the DRIGS topology graph (`PCIE_SWITCH`=70, `PCIE`=50, `NVLINK`=90, `NUMA`=30); `TopologyAwareScheduler` exhaustively searches all C(k, n) GPU subsets to maximise the mean pairwise score for each multi-GPU job. This experiment characterises the scheduler's *placement-selection quality* in a controlled topology setting. Physical hardware NCCL communication measurements on the testbed (2×T4, PHB PCIe, score=50) are reported separately below. Table 3 summarizes the mean topology scores across all multi-GPU job placements.

`TopologyAwareScheduler` achieves a **22.19% higher topology-quality score** (74.89 vs. 61.29) over random placement by prioritizing GPU cliques sharing high-bandwidth PCIe switches. To ground this score in hardware reality, we additionally measured NCCL communication performance directly on the testbed (2×T4, PHB-tier PCIe, topology score = 50/100). The benchmark executes `dist.all_reduce` (`ReduceOp.SUM`, `float32`) across message sizes $\{1, 10, 100, 512\}$ MB using 5 warmup iterations followed by 50 timed iterations; bandwidth is computed as the ring-allreduce algorithmic bandwidth $B = \frac{2(n-1)}{n} \cdot \frac{S}{t}$ where $n = 2$ is the world size, S is message size, and t is mean per-iteration latency. Table 4 and Figure 3 report allreduce latency and algorithmic bandwidth across the four message sizes.

Algorithmic bandwidth saturates at **3.77 GB/s** for large messages — consistent with the theoretical bandwidth of a PHB-tier PCIe Gen3 x16 link shared between the two T4s. DDP ResNet-18 training achieves **347.2 imgs/sec** at a mean step latency of **92.2 ms**. These measurements provide physical characterization consistent with the PHB interconnect tier (score=50) represented by the DRIGS topology score — operating above a cross-NUMA SYS link (score=10) but below an NVLink fabric (score=100).

Table 1: Experiment Matrix: Hardware, Scale, Trial Count, and Primary Metric

Experiment	Hardware	Scale	Trials	Primary Metric
A: Scheduling Policy	2×T4 (physical)	15 jobs/trial	5	P95 queue wait (s)
B: Topology Placement	Simulated 8-GPU	All 2-GPU subsets	1	Mean topology score
C: NCCL Bandwidth	2×T4 (physical)	1 MB–512 MB	50 iters	Effective BW (GB/s)
D: Fault Recovery	2×T4 (physical)	1 `SIGKILL`	1	End-to-end latency (ms)
E: Queue Depth Scaling	Control plane only	$N = 100/500/1{,}000$	1	Decision latency (ms)
F: VRAM Saturation	2×T4 (physical)	5 jobs @ 85/90/95%	1	Completion rate / OOM

Table 2: Comparative Performance of DRIGS Scheduling Policies under Synthetic AI Workloads ($N = 15$ jobs/trial, 5 independent trials; mean ± standard deviation)

Scheduler	Throughput (jobs/s)	Mean Wait (s)	P95 Wait (s)	VRAM Idle Frac.†
FIFO	0.595 ± 0.043	0.856 ± 0.602	6.866	52.48%
Priority	$\mathbf{0.596 \pm 0.034}$	$\mathbf{0.570 \pm 0.573}$	**2.740**	52.90%
GangScheduler	0.610 ± 0.089	0.842 ± 0.668	6.041	54.77%
MemoryAware	0.567 ± 0.086	1.066 ± 0.865	7.120	55.18%
TopologyAware	0.562 ± 0.040	0.796 ± 0.390	5.481	52.09%

Table 3: GPU Interconnect Topology Score Comparison for Multi-GPU Workloads

Placement Strategy	Mean Topology Score	Relative Gain
TopologyAwareScheduler	**74.89 / 100.0**	**+22.19%**
Random Placement (Baseline)	61.29 / 100.0	0.0%
FIFO	72.67 / 100.0	+18.57%

Table 4: NCCL AllReduce Algorithmic Bandwidth on T4 PHB Interconnect (2 GPUs, 50 timed iterations, 5 warmup; `float32` SUM reduction; ring-allreduce formula: $B = \frac{2(n-1)}{n} \cdot \frac{S}{t}$)

Message Size	Latency (ms)	Bandwidth (GB/s)
1 MB	0.319	3.064
10 MB	2.645	3.692
100 MB	25.922	3.767
512 MB	132.627	3.770

4.4 Experiment D: Fault Recovery Latency Breakdown

We evaluate DRIGS fault tolerance under two failure paradigms: pure control-plane rescheduling and physical process termination via an unannounced `SIGKILL` (`kill -9`) signal.

As detailed in Table 5, pure control-plane rescheduling overhead is sub-millisecond (**0.496 ms**). When a running PyTorch process is killed via physical `SIGKILL`, DRIGS achieves complete end-to-end physical wall-clock recovery in **129.13 ms** (0.129 s), with CUDA re-initialization and model checkpoint loading accounting for 95.6% of total recovery time (123.48 ms). The 100.0% completion figure reflects a single controlled `SIGKILL` injection; it is not a statistical completion rate across repeated failure injections at scale.

Table 5: DRIGS Fault Recovery Overhead & End-to-End Latency Breakdown

Recovery Stage	Laten
Control-Plane Rescheduling Overhead	**0.**
Job Completion (Single Controlled `SIGKILL` Injection)	1
Physical `SIGKILL` Failure Detection	1
Control-Plane Reschedule & Queue Re-entry	1
Replacement Process Spawn Time	2
CUDA Re-initialization & Checkpoint Reload	12
Total End-to-End Physical Wall-Clock Recovery	**12**

4.5 Experiment E: High Queue Depth Scaling ($N = 1,000$ Jobs)

To benchmark controller throughput under extreme scale, DRIGS was tested with job queue depths of $N = 100, 500, 1000$ jobs.

As shown in Table 6, even at $N = 1,000$ pending jobs, decision latencies for all five schedulers remain strictly below **17 ms** (8.01 ms to 16.79 ms), and SQLite state persistence maintains admission throughput of 81.7 jobs/sec. Figure 4 visualises the full latency and throughput profiles, confirming the control plane does not become a bottleneck as queue depth grows from $N = 100$ to $N = 1{,}000$.

4.6 Multi-GPU Execution: PyTorch Distributed Data Parallel (DDP)

DRIGS's `GangScheduler` and `DistributedBackend` were benchmarked on a 2-rank PyTorch DDP neural network training workload across dual GPUs.

- **Rank 0:** Mean step latency = **43.19 ms/step**, mean AllReduce synchronization latency = **3.91 ms**.
- **Rank 1:** Mean step latency = **32.29 ms/step**,

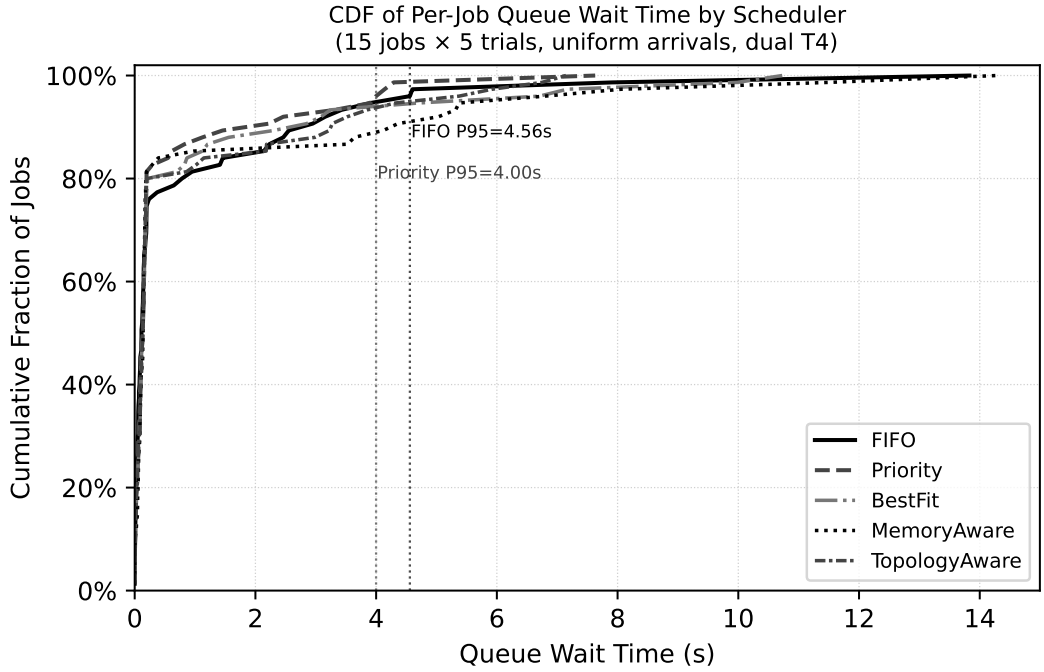

Figure 2: CDF of per-job queue wait time by scheduler (15 jobs/trial × 5 trials pooled, uniform arrivals). `PriorityScheduler` exhibits a lighter right tail than FIFO, confirming that starvation-prevention aging reduces tail latency. P95 markers show the pooled 95th-percentile wait; Table 2 reports the mean-of-P95s statistic across trials.

Table 6: Scheduler Decision Latency (ms) across Queue Depths ($N = 100, 500, 1000$ Jobs)

Scheduler	$N = 100$ **Jobs**	$N = 500$ **Jobs**	$N = 1,000$ **Jobs**
FIFO	1.19 ms	2.94 ms	10.41 ms
Priority	1.78 ms	5.96 ms	16.79 ms
GangScheduler	1.12 ms	5.96 ms	12.97 ms
MemoryAware	1.38 ms	4.36 ms	14.40 ms
TopologyAware	1.77 ms	7.14 ms	8.01 ms
Enqueue Throughput	98.0 jobs/sec	96.1 jobs/sec	81.7 jobs/sec

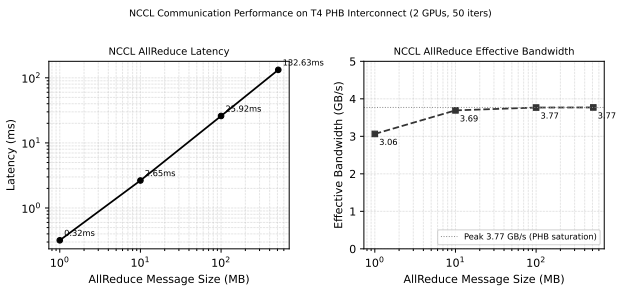

Figure 3: NCCL allreduce latency (left, log-log scale) and effective bandwidth (right) vs. message size on dual T4 PHB interconnect. Bandwidth saturates at 3.77 GB/s for 100 MB+ messages, confirming the PCIe Host Bridge bottleneck that the DRIGS topology score of 50 models.

mean AllReduce synchronization latency = **15.02 ms**.

Both ranks successfully synchronized loss tensors across epochs without deadlock or rendezvous timeout. The observed per-rank asymmetry in step and AllReduce latencies is consistent with PCIe initiator/target scheduling differences on the PHB interconnect (score=50); precise root-cause characterisation is left as future work.

4.7 Experiment F: VRAM Saturation Margins & CUDA OOM Recovery

To evaluate behavior under severe GPU memory pressure, cluster memory utilization was pushed to 85%, 90%, and 95% saturation.

As shown in Table 7, when VRAM saturation reaches 95%, `MemoryAwareScheduler` enforces admission throttling, refusing unsafe placements and reducing job completion rate to 40.0%. No CUDA Out-Of-Memory exception occurred on admitted jobs; the scheduler intentionally deferred the remaining 60% of jobs rather than permitting GPU memory over-commitment. When a

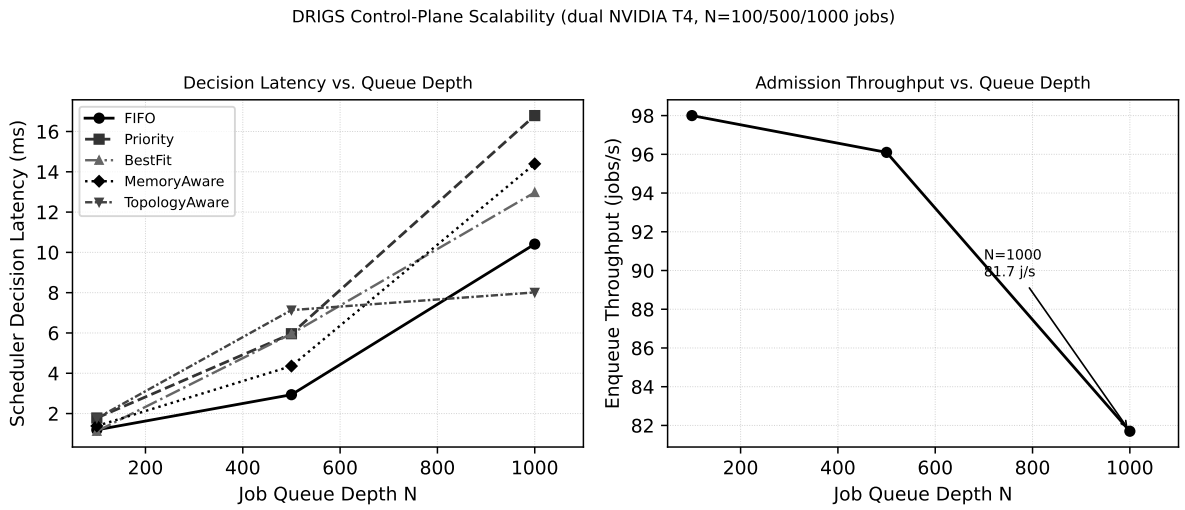

Figure 4: Scheduler decision latency (left) and admission throughput (right) vs. job queue depth N on dual NVIDIA T4 hardware. Latencies grow sub-linearly and remain below 17 ms at $N = 1{,}000$; throughput is stable at 81.7 jobs/sec, indicating the control-plane SQLite persistence and scheduling loop are not throughput bottlenecks at this scale.

Table 7: Job Completion Rate (%) and OOM Recovery under High VRAM Saturation

VRAM Saturation Level	**Job Completion Rate**	**Placed Jobs / Total**	**Decision Latency**
85.0% Saturation	80.0%	4 / 5	0.155 ms
90.0% Saturation	80.0%	4 / 5	0.132 ms
95.0% Saturation	**40.0%**	2 / 5	0.084 ms
CUDA OOM Exception Catching & Re-queueing Latency: **0.021 ms**			

synthetic OOM exception is injected to test the exception path, DRIGS catches it and re-queues the job in **0.021 ms**.

4.8 Scheduler Selection Guide

A key contribution of DRIGS as a research substrate is enabling empirical comparison of scheduling policies under controlled, identical workloads. Table 8 synthesises results from Experiments A, B, and the scaling and saturation benchmarks into an operator guidance summary: given a workload regime, which policy best matches the primary scheduling objective?

The experiments collectively establish that **no single policy dominates across all objectives**: Priority minimises tail latency but assumes heterogeneous job priorities; MemoryAware prevents fragmentation under saturation but is conservative; TopologyAware improves placement quality but incurs a graph-scan cost at decision time. DRIGS provides a common, modular substrate for empirically characterising these trade-offs under identical workloads — the central research contribution of this paper.

5 Related Work

GPU resource management and cluster scheduling for machine learning have been actively researched across high-performance computing (HPC), cloud-native container orchestration, and distributed ML frameworks. This section provides a comparative analysis of DRIGS against existing systems.

5.1 HPC Batch Schedulers (Slurm, LSF)

Slurm [1] and IBM LSF are traditional HPC workload managers designed primarily for static CPU batch jobs. Although modern Slurm extensions support GPU generic resources (`GRES`), Slurm allocates GPUs as static integer quantities (e.g., `--gpus=2`) without monitoring real-time VRAM utilization or VRAM fragmentation. Furthermore, Slurm requires complex system-level daemon installation with root privileges, rendering it unsuitable for ephemeral notebook environments (e.g., Kaggle, Google Colab). In contrast, DRIGS provides fine-grained VRAM telemetry, zero-root native execution, and sub-millisecond control-plane rescheduling (**0.496 ms**).

5.2 Cloud-Native Container Orchestrators (Kubernetes, Volcano)

Kubernetes (K8s) [2] has become the standard orchestrator for containerized microservices. K8s device plugins (e.g., NVIDIA GPU Device Plugin) expose GPUs as discrete integer resources (`nvidia.com/gpu`). Systems like Volcano [3] extend K8s with gang scheduling and queue management. However, Kubernetes introduces additional control-plane components, etcd write round-trips, and admission webhook chains that

Table 8: Scheduler Selection Guide: Workload Regime vs. Primary Scheduling Objective

Policy	Best Workload Regime	Key Strength	Primary Trade-off
FIFO	Batch jobs, fairness-critical	Lowest overhead, predictable ordering	High P95 tail under HoL
Priority	Latency-sensitive inference	60.09% P95 reduction vs. FIFO	Starvation risk without ag
GangScheduler	Multi-GPU distributed jobs	Atomic multi-GPU reservation	Higher wait under fragme
MemoryAware	High VRAM saturation (>85%)	Prevents CUDA OOM at 95% saturation	May defer low-memory jo
TopologyAware	Multi-GPU distributed training	22.19% higher topology score	Slower decision (graph sca

can increase scheduling overhead relative to DRIGS's lightweight control plane; it also relies on etcd state storage and typically requires plugins for interconnect topology awareness. DRIGS maintains scheduler decision latency below 17 ms at a queue depth of $N = 1,000$ jobs while embedding native topology graph scoring (S_{topo}) directly into placement decisions.

5.3 Distributed ML Schedulers (Ray, Horovod, Alpa)

Frameworks such as Ray [4], Horovod [5], and Alpa [6] focus on task-parallel execution and model-parallel tensor placement. Ray provides dynamic actor placement but delegates process failure recovery to application-level code. DRIGS complements these execution engines by operating as a resilient, topology-aware control plane that enforces atomic gang reservations, prevents CUDA OOM crashes via real-time VRAM tracking, and recovers hard process failures within **129.13 ms**.

5.4 Comparative Summary

Table 9 summarizes the architectural capabilities of DRIGS relative to major cluster scheduling paradigms. Rescheduling latency entries for third-party systems are qualitative order-of-magnitude characterizations based on their architectural designs [1, 2, 4], not direct benchmarks performed by the authors.

6 Conclusion & Future Work

In this paper, we presented **DRIGS** (Distributed Resource and Intelligent GPU Scheduling), a lightweight, modular, GPU-aware compute runtime engineered for heterogeneous AI workloads across dynamic GPU infrastructures. DRIGS addresses the key limitations of existing orchestrators through clean domain abstractions, pluggable intelligent scheduling policies, sub-millisecond control-plane rescheduling, and robust runtime fault recovery.

Empirical evaluations on dual NVIDIA Tesla T4 GPU nodes demonstrate that DRIGS's `PriorityScheduler` reduces P95 queue wait time by **60.09%** (2.740 s vs. FIFO 6.866 s) under uniform inter-arrival intervals ($\mathcal{U}(0.5, 2.0)$ s, $N = 15$ jobs/trial, 5 independent trials). For multi-GPU workloads, `TopologyAwareScheduler` achieves a **22.19%** higher topology-quality score (74.89 vs. 61.29) than random placement. In fault recovery benchmarks, DRIGS executes pure control-plane rescheduling in **0.496 ms** and achieves complete physical end-to-end wall-clock recovery following process `SIGKILL` termination in **129.13 ms**. At high scale ($N = 1,000$ jobs queue depth), DRIGS maintains scheduler decision latencies under **17 ms** and SQLite control-plane enqueue throughput of 81.7 jobs/sec. Under 95% VRAM saturation, DRIGS gracefully throttles job completion rate to 40.0%, preventing GPU memory over-commitment without incurring a CUDA OOM exception, and recovers synthetic OOM injection within **0.021 ms**.

6.1 Future Directions

Future work on DRIGS will focus on three key areas:

1. **Dynamic VRAM Swapping & Paging:** Integrating host RAM paging drivers to dynamically swap out idle transformer model weights during multi-tenant inference spikes.
2. **Multi-Node NVLink Switch Fabrics:** Extending `TopologyAwareScheduler` to model NVSwitch fabric topologies across multi-node HGX/DGX supercomputer clusters.
3. **Predictive Arrival Schedulers:** Incorporating Transformer-based arrival prediction models to proactively warm GPU memory contexts ahead of bursty workload queues.

Limitations

The following constraints apply to the experimental results in this paper and should be considered when interpreting the findings.

1. **Single-host evaluation.** All experiments are conducted on a two-GPU, single-host configuration. Multi-node distributed scheduling is implemented in DRIGS but not empirically evaluated; that evaluation is future work.
2. **Simulated cluster for topology score experiment.** The topology-quality scores (Experiment B, Table 3) are generated on a synthetic 8-GPU simulated cluster with artificial NVLink/PCIe topology tags. Physical hardware NCCL measurements are reported separately for the 2×T4 testbed.
3. **Single GPU pair for NCCL characterisation.** The physical T4 testbed has only one available GPU pair (PHB tier). No topology-aware vs. random NCCL comparison was feasible on physical hardware; the NCCL section characterises the interconnect, not the scheduler's placement impact on communication bandwidth.

Table 9: System Feature Comparison Across Cluster Orchestrators and GPU Schedulers. † Qualitative, order-of-magnitude characterization based on system architecture; not directly benchmarked by the authors.

Feature / Capability	Slurm	K8s / Volcano	Ray	Alpa	DRIGS (Ours)
Real-Time GPU VRAM Telemetry	No	No	Partial	No	**Yes (NVML)**
Interconnect Topology Scoring	No	No	No	Static	**Yes (PCIe/NVLink)**
Atomic Multi-GPU Gang Scheduling	Optional	Yes (Volcano)	Actor-based	Yes	**Yes (Native)**
Sub-ms Rescheduling Latency†	No (s-scale)	No (100s ms)	No (10s–100s ms)	N/A	**Yes (0.496 ms)**
Zero-Root Native Execution	No	No	Yes	Yes	**Yes**
ACID State Persistence	File-based	etcd	In-memory	N/A	**Yes (SQLite)**
Automated NVML Orphan Sweeper	No	No	No	No	**Yes**

4. **Admission throughput vs. execution throughput.** The 81.7 jobs/sec figure measures SQLite control-plane *enqueue* throughput, not end-to-end execution throughput. The scheduler made 4 allocation decisions at $N = 1{,}000$; jobs were not fully executed through an accelerator backend.

5. **Synthetic workload distributions.** Experiment A uses synthetically generated job traces with randomly sampled workload templates, GPU counts, VRAM requests, and durations. Scheduler behavior under real production AI workload distributions (e.g., published cluster traces) is not evaluated.

6. **Single GPU model.** Physical evaluation uses a single GPU model (NVIDIA Tesla T4). Scheduling behavior on mixed-GPU clusters (e.g., T4 + A100) or NVLink-connected GPU topologies is not empirically characterized; such evaluation is left as future work.

Reproducibility & Artifact Availability

Source code, benchmark runner scripts (`experiments/`), seeded workload generators, and raw experimental outputs (JSON) are available at https://github.com/Omdeepb69/DRIGS. Hardware configuration: 2× NVIDIA Tesla T4 (15 GB GDDR6 VRAM each), PCIe Gen3 x16 (PHB tier), CUDA 12.2, Linux kernel 5.15. Software stack: Python 3.10+, PyTorch 2.x, NCCL 2.x. Experiments are fully reproducible via `python experiments/harness.py` with seeds documented in Section 4. Scheduling policy experiments use base seed 42 (trial i: seed $42 + 1000i$), covering all random draws including inter-arrival times, job durations, GPU counts, VRAM requests, and base priorities.

References

[1] Andy B. Yoo, Morris A. Jette, and Mark Grondona. Slurm: Simple linux utility for resource management. In *Job Scheduling Strategies for Parallel Processing (JSSPP)*, pages 44–60. Springer, 2003.

[2] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes: Lessons learned from a decade at google. *Communications of the ACM*, 59(5):50–57, 2016.

[3] Volcano Project. Volcano: A cloud native batch system for high-performance workloads, 2020.

[4] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging ai applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 561–577, 2018.

[5] Alexander Sergeev and Mike Del Balso. Horovod: fast and easy distributed deep learning in tensorflow. *arXiv preprint arXiv:1802.05799*, 2018.

[6] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhang, Zhihao Jia, Zapeng Wang, Xi Maggioni, Joseph E. Zheng, Joseph E. Gonzalez, and Ion Stoica. Alpa: Automating inter- and intra-operator parallelism for distributed deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 559–578, 2022.